Aggregates

A wide range of aggregate techniques, including **`count`**, **`max`**, **`min`**, **`average`**, and **`total`**, are now available in the query builder.

Count:

```
$employees = DB::table('employees')->count();
```

Max:

```
$cost = DB::table('orders')->max('cost');
```

Min:

```
$cost = DB::table('orders')->min('cost');
```

Average:

```
$cost = DB::table('orders')->avg('cost');
```

Sum:

```
$total = DB::table('employees')->sum('votes');
```

Raw Expressions

A query may occasionally need the usage of a raw expression. Take caution not to construct any SQL injection points because these expressions will be injected into the query as strings! You can use the **`DB::raw`** command to produce a raw expression:

```
$employees = DB::table('employees')
       ->select(DB::raw('count(*) as employee_count, status'))
       ->where('status', '<>', 1)
       ->groupBy('status')
       ->get();
```

Pessimistic Locking

There are a few functions in the query builder that will assist you with **pessimistic locking** for your **`SELECT`** statements.

Use the **`sharedLock`** method on a query to execute the **`SELECT`** statement with a **shared lock**:

```
DB::table('employees')->where('votes', '>', 140)->sharedLock()->get();
```